CCNA 200-301 V2.0

10 Trunking, 802.1Q and SVIs

Part II — Switching and Network Access

EXAM TOPICS

2.1

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Explain** what an 802.1Q tag is and why the native VLAN is the exception that causes most trunk faults.
- **Configure** a trunk between two switches and between a switch and a router, and prune it to the VLANs it should carry.
- **Configure** switch virtual interfaces so that a multilayer switch routes between VLANs.
- **Diagnose** the four trunk faults: mode mismatch, encapsulation mismatch, native VLAN mismatch and a missing VLAN in the allowed list.

ASSUMED KNOWLEDGE

Before we start

Chapter 8 for VLANs and access ports; Chapter 4 for addressing, which the SVI section needs.

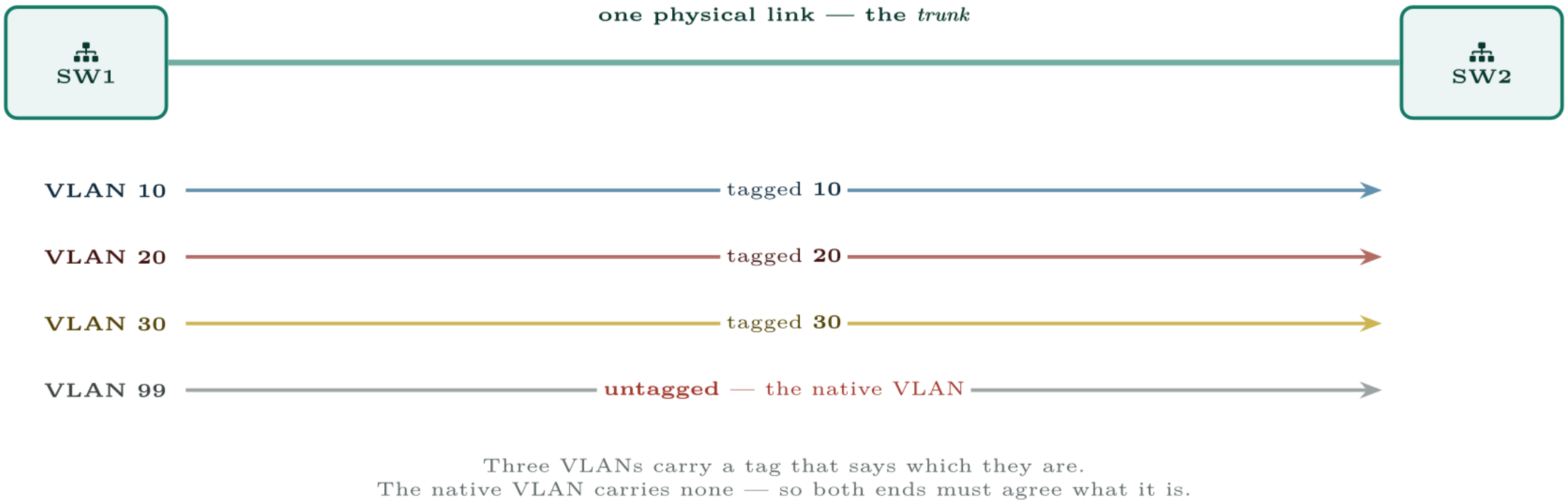
KEY TERMS

Vocabulary for this chapter

- trunk
- 802.1Q
- tag
- native VLAN
- allowed VLAN list
- DTP
- dynamic auto
- dynamic desirable
- SVI
- inter-VLAN routing
- router on a stick
- subinterface

Each is defined where it first matters — not here.

The tag, and the one VLAN that has none



Three VLANs carry a tag that says which they are.
The native VLAN carries none — so both ends must agree what it is.

A trunk is one cable carrying many VLANs. Every VLAN but one is labelled; the unlabelled one is the native VLAN, and a disagreement about it joins two networks silently.

CONCEPT

The native VLAN is untagged, and that is the whole problem

One VLAN on every trunk is designated the **native VLAN**, and its frames cross the trunk **untagged**. The receiving switch assumes any untagged frame it sees on a trunk belongs to *its own* native VLAN.

If the two ends disagree about which VLAN is native, traffic silently leaks between VLANs: frames sent untagged in VLAN 1 by one switch are received into VLAN 99 by the other. Nothing goes down, nothing logs an error at layer 2, and two VLANs that should be isolated are now joined. CDP will complain, which is one of several reasons to leave it on (Chapter 13).

Set the native VLAN explicitly and identically at both ends, and make it an unused VLAN — never VLAN 1, and never a VLAN carrying users.

CONFIGURE

A trunk between two switches, configured deliberately

Configure

```
! On BOTH ends, identically.
interface GigabitEthernet0/1
 description Uplink to SW2
 ! Some platforms require the encapsulation before the mode; on
 ! switches that only support dot1q the command is rejected as
 ! unavailable, which is not an error.
 switchport trunk encapsulation dot1q
 switchport mode trunk
 ! An unused VLAN as native, matched at the far end.
 switchport trunk native vlan 999
 ! Carry only what this link needs. Pruning limits broadcast
 ! flooding and limits the blast radius of a mistake.
 switchport trunk allowed vlan 10,20,30
 ! Do not negotiate -- we have decided what this port is.
 switchport nonegotiate
 no shutdown
```

VERIFY

show interfaces trunk

```
SW1#  show interfaces trunk

Port        Mode         Encapsulation  Status        Native vlan
Gi0/1       on           802.1q         trunking      999

Port        Vlans allowed on trunk
Gi0/1       10,20,30

Port        Vlans allowed and active in management domain
Gi0/1       10,20,30

Port        Vlans in spanning tree forwarding state and not pruned
Gi0/1       10,20,30
```

WATCH OUT

allowed vlan replaces; allowed vlan add appends

Typing `switchport trunk allowed vlan 40` on a live trunk does not add VLAN 40 — it makes 40 the *only* allowed VLAN and instantly drops every other VLAN on that link. On a production uplink this is an outage.

Use `switchport trunk allowed vlan add 40`. The equivalents are `remove`, `except` and `all`.

CONCEPT

Configure both ends; negotiate nothing

Set `switchport mode trunk` and `switchport nonegotiate` on uplinks, and `switchport mode access` on edge ports. Then a port is what you said it is, and an attacker who plugs a laptop into a wall socket cannot negotiate himself a trunk into every VLAN.

CONFIGURE

A multilayer switch routing three vlans

Configure

```
! Routing must be switched on -- easily forgotten, and without it the
! SVIs come up and nothing is routed between them.
ip routing
!
interface Vlan10
 description Staff gateway
 ip address 192.168.10.1 255.255.255.0
 no shutdown
!
interface Vlan20
 description Guest gateway
 ip address 192.168.20.1 255.255.255.0
 no shutdown
!
interface Vlan30
 description Voice gateway
 ip address 192.168.30.1 255.255.255.0
 no shutdown
```

WATCH OUT

What it takes to bring an SVI up

An SVI shows `up/up` only when *all* of the following hold:

- the VLAN exists in the VLAN database and is not shut down;
- at least one access port in that VLAN is up, **or** the VLAN is allowed and forwarding on a trunk;
- the SVI itself is not shut down.

An SVI that stays down with a perfectly good address almost always means the second condition: no live port in that VLAN. It is not an addressing fault.

VERIFY

show ip interface brief | include Vlan

```
SW1#  show ip interface brief | include Vlan
Vlan10                 192.168.10.1    YES manual up                    up
Vlan20                 192.168.20.1    YES manual up                    up
Vlan30                 192.168.30.1    YES manual up                    down
```

Router on a stick

Configure

```
interface GigabitEthernet0/0
 description Trunk to SW1
 no ip address
 no shutdown
!
interface GigabitEthernet0/0.10
 description Staff
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
!
interface GigabitEthernet0/0.20
 description Guest
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
!
! The native VLAN's subinterface is marked, not tagged.
interface GigabitEthernet0/0.999
 encapsulation dot1Q 999 native
```

Router on a stick — the older way

- **Native VLAN mismatch.** The two ends disagree, VLANs leak, and nothing goes down. CDP logs it; look for `%CDP-4-NATIVE_VLAN_MISMATCH`.
- **Bare `allowed vlan` on a live trunk.** Replaces the list. Always `add`.
- **Forgetting `ip routing` on a multilayer switch.** The SVIs come up and traffic still will not cross between VLANs.
- **Expecting an SVI to be up with no active port in its VLAN.** It will not be, and no amount of re-addressing changes that.
- **Leaving VLAN 1 as native.** It carries control traffic and it is every attacker's first guess.

One VLAN crosses the uplink; the other three do not.

After a maintenance window, users in VLAN 10 are fine and users in 20, 30 and 40 cannot reach the servers. Both uplink ports show as trunking.

One VLAN crosses the uplink; the other three do not.

What would you check first — and what do you expect to see?

show interfaces trunk

```
SW2#  show interfaces trunk

Port        Mode         Encapsulation  Status        Native vlan
Gi0/1       on           802.1q         trunking      999

Port        Vlans allowed on trunk
Gi0/1       10
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. The trunk is healthy; the allowed list has been reduced to VLAN 10 alone. Somebody ran `switchport trunk allowed vlan 10` intending to add VLAN 10 to the existing list, and replaced it instead. The link stayed up throughout, which is why nothing alerted.

And the lesson

Fix. Restore the full list explicitly rather than adding one VLAN at a time: `switchport trunk allowed vlan 10,20,30,40`. Verify with `show interfaces trunk` on *both* ends — the allowed list is configured per side, and the effective set is the intersection, so a matching mistake at the far end would leave the fault half-fixed and twice as confusing.

LAB · PACKET TRACER

Trunk two switches and route between VLANs

Topology. Two switches trunked together; a multilayer switch or router providing gateways; two PCs per VLAN across both switches.

1. Create VLANs 10, 20 and 999 on both switches. Put PCs in 10 and 20 on each switch.
2. Before configuring the trunk, try to ping between the two switches' VLAN 10 PCs. Record the failure and explain it.
3. Configure the trunk on both ends with native VLAN 999 and an allowed list of 10, 20 and 999. Re-test.
4. Change the native VLAN on one end only to 998. Record what still works, what breaks, and what the log says.
5. Restore it, then run `switchport trunk allowed vlan 20` on one end and observe the outage you have just caused. Fix it with `add`.

Trunk two switches and route between VLANs (continued)

1. Enable `ip routing` and configure SVIs for VLANs 10 and 20. Prove inter-VLAN routing works.
2. **Extension.** Shut every access port in VLAN 20 and watch the SVI go down. Explain why in terms of the three conditions above.

CHECKPOINT 1 of 3

Both ends of a link are set to `dynamic` `auto`. What does the link become, and why?

Discuss · then advance

ANSWER

Checkpoint 1 of 3

Both ends of a link are set to `dynamic auto`. What does the link become, and why?

> An access port. `dynamic auto` will accept a trunk if asked but never asks, so with neither side initiating, no trunk forms.

CHECKPOINT 2 of 3

An SVI has a correct address and shows up/down. What is the most likely cause and how would you confirm it?

Discuss · then advance

ANSWER

Checkpoint 2 of 3

An SVI has a correct address and shows `up/down`. What is the most likely cause and how would you confirm it?

> No access port in that VLAN is up, so the SVI has nothing to be up for. Confirm with `show vlan brief` — an SVI needs the VLAN to exist and at least one active port in it (or `no autostate`).

CHECKPOINT 3 of 3

Why is a native VLAN mismatch more dangerous than a trunk that fails to come up at all?

Discuss · then advance

ANSWER

Checkpoint 3 of 3

Why is a native VLAN mismatch more dangerous than a trunk that fails to come up at all?

> Because a trunk that fails to come up is obvious and fails safe — no traffic passes and somebody investigates. A native VLAN mismatch leaves the trunk up and quietly bridges two different VLANs together, merging broadcast domains that were meant to be separate. It is a security problem that looks like a working link.

What to take away

- 802.1Q tags every VLAN except the native one, which crosses untagged. Mismatched native VLANs leak traffic between VLANs silently.
- Configure both ends explicitly and add `switchport nonegotiate`. `dynamic auto` on both ends yields an access port, not a trunk.
- `allowed vlan` replaces the list; `allowed vlan add` appends. This distinction causes real outages.
- `show interfaces trunk` narrows from allowed to active to forwarding; the block where a VLAN disappears tells you the layer at fault.

What to take away (continued)

- An SVI needs `ip routing`, an existing unshut VLAN, and at least one live port in that VLAN.

Where we got to

- 802.1Q tags every VLAN except the native one, which crosses untagged. Mismatched native VLANs leak traffic between VLANs silently.
- Configure both ends explicitly and add `switchport nonegotiate`. `dynamic auto` on both ends yields an access port, not a trunk.
- `allowed vlan` replaces the list; `allowed vlan add` appends. This distinction causes real outages.
- `show interfaces trunk` narrows from allowed to active to forwarding; the block where a VLAN disappears tells you the layer at fault.

NEXT

Chapter 11 — EtherChannel and LACP